sudo_docs Template

for beautiful computer science notes

John Smith, Jane Doe

example university

A.Y. 2025-2026

Contents

1. Introduction

Welcome to **sudo_docs**, a minimalist, cs-centric Typst template designed for Computer Science students who also happen to like having clean, colorful notes.

This document demonstrates the visual style and capabilities of the template. The font used for the text is **IBM Plex Mono**, giving it a distinctive "terminal" look, while the code blocks use **Fira Code** (or **Cascadia Code**) for better readability.

1.1. How to use this template

To use this template, simply import it at the top of your `.typ` file:

```
// Use * to import the project function, img, and the custom cards
#import "sudodocs.typ": *

#show: project.with(
  title: "My Project",
  author: "My Name",
  toc: true,
  is-appendix: false, // set to true to reset counters for appendices
  main-color: rgb("#4d1d14") // optional custom color
  lang: "en" // "en" for English, "it" for Italian
)
```

The template natively supports multiple languages (currently English "en" and Italian "it"). By changing the lang parameter in the project setup, Typst will automatically adjust hyphenation rules, translate default elements (such as turning "Table of Contents" into "Indice"), and seamlessly translate all the internal labels of the CS "Identity Cards" (`#adt-card` and `#algo-card`).

1.1.1. Table of Contents & Sections

If you have a large document and want your Table of Contents to stop before a certain section (like an Appendix), simply write the `<new-section>` label anywhere in your file where the main content ends. The TOC will automatically stop tracking headings past that point. If you want to create a dedicated appendix document with its own TOC and reset page numbers, you can just set `is-appendix: true` in the project configuration.

1.2. Lists

The lists are styled according to the main color of the template, that is customable at line 13 through a RGB code.

- First item
- Second item
- Third item

1. First item
2. Second item
3. Third item

1.3. Code

Here is an example of inline code: var x int. For code blocks:

```
func main() {
    fmt.Println("hello world")
}
```

The code block has a stroke that matches the main color of the template that you can modify.

1.4. Image alignment

This is a short tutorial on how to use the img function:

The image automatically centers if you don't specify any parameters;

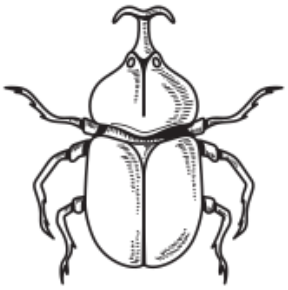
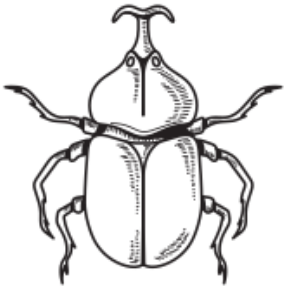
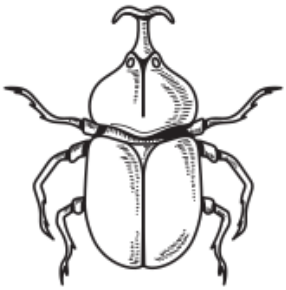


Figure 1:
centered image

You can align an image to the left using the pos (position) and width parameters. You can define size and position easily instead of using typst functions easily.





Description for images is only available for centered images due to space requirements.

This is an easy way to add images of graphs, data or add specific smaller images such as icons or small doodles from your classes.

1.5. CS “Identity Cards” (ADT & Algorithms)

This template includes two special functions designed specifically for Computer Science notes: `adt-card` and `algo-card`. They create beautiful, structured summary boxes that automatically match your main-color for data structures and algorithms. You can create your own by forking this template’s repo and modifying the lib.typ file and changing this function’s name and parameters.

1.5.1. Abstract Data Type (ADT) Card

Use this to summarize Data Structures. All parameters except name are optional. You can easily embed visuals using the image parameter alongside the custom img function. Snippet di codice

Dictionary (Map / Hash Table)

Abstract Data Type

What it represents:
A collection of **key-value** pairs, where each key is unique.

Implementation methods:
- Hash Tables
- Balanced Binary Search Trees

Main operations / functions:
- `insert(k, v)`: Inserts a pair.
- `search(k)`: Returns the value.

```
#adt-card(
  name: "Dictionary (Map / Hash Table)",
  desc: [A collection of *__key-value__* pairs, where each key is unique.],
  image: img(image("logo.png"), width: 6cm), // Adds a nice picture inside the card!
  impl: [
    - Hash Tables
    - Balanced Binary Search Trees
  ],
  funcs: [
    - `insert(k, v)`: Inserts a pair.
    - `search(k)`: Returns the value.
  ]
)
```

1.5.2. Algorithm Card

Use this for Algorithms. It automatically creates a 2-column grid for complexity and use-cases to save vertical space, and beautifully formats your pseudo code blocks. Snippet di codice

Merge Sort

Algorithm

Description:
A stable sorting algorithm based on the **Divide and Conquer** paradigm.

How it works:
1. **Divide:** Split the array in half.
2. **Conquer:** Solve sub-problems.
3. **Combine:** Merge sub-arrays.

Complexity:
- **Time:** $O(n \log n)$
- **Space:** $O(n)$

Use cases:
- Sorting Linked Lists
- External Sorting

Pseudocode:

```
function MergeSort(A):
  if length(A) <= 1:
    return A
  // ... recursive logic here
```

```
#algo-card(
  name: "Merge Sort",
  desc: [A stable sorting algorithm based on the *Divide and Conquer* paradigm.],
  working: [
    1. *Divide:* Split the array in half.
    2. *Conquer:* Solve sub-problems.
    3. *Combine:* Merge sub-arrays.
  ],
  complexity: [
    - *Time:* $O(n log n)$
    - *Space:* $O(n)$
  ],
  use-cases: [
    - Sorting Linked Lists
    - External Sorting
  ],
  pseudo: [
```

```
python function MergeSort(A): if length(A) <= 1: return A
```

```
  ]
)
```

2. Done!

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.